

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №1

«Графы.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 22

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Реализовать структуру данных «взвешенный неориентированный граф» с использованием матрицы смежности
2. Разработать алгоритмы:
 - a) Обхода графа в ширину (BFS)
 - b) Обхода графа в глубину (DFS)
 - c) Поиска кратчайших путей от одной вершины до всех остальных (алгоритм Дейкстры)
 - d) Поиска кратчайших путей между всеми парами вершин (алгоритм Флойда-Уоршелла)
3. Создать графический интерфейс пользователя для взаимодействия с программой
4. Реализовать пошаговую визуализацию работы каждого алгоритма с возможностью просмотра промежуточных результатов
5. Обеспечить возможность редактирования графа (добавление/удаление вершин и рёбер, изменение весов)
6. Обеспечить возможность редактирования матрицы смежности

АНАЛИЗ.

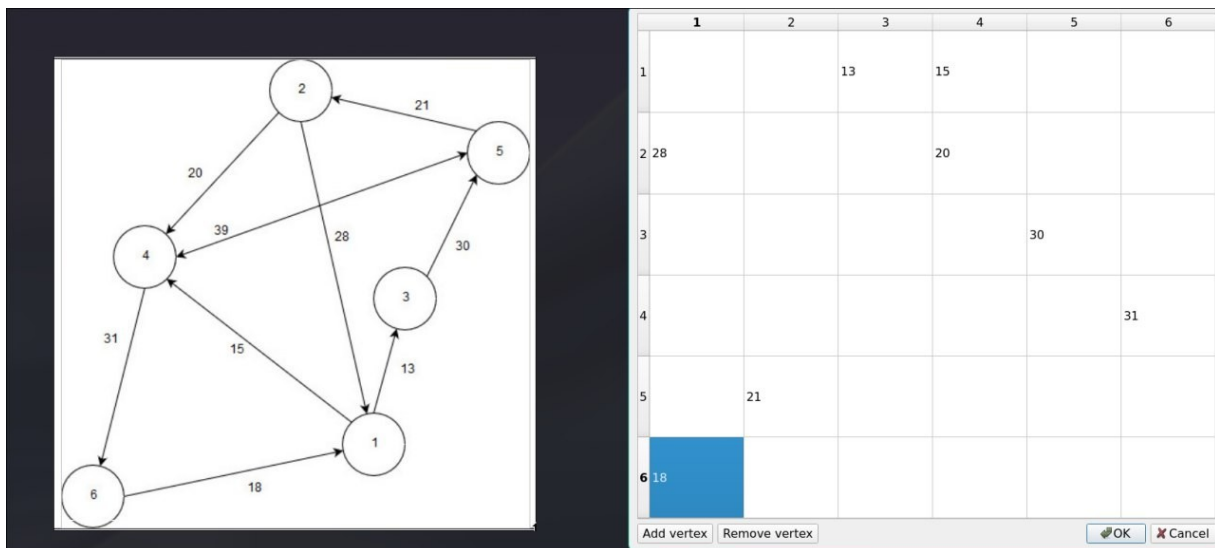
Визуализация графа обеспечивается через UiVertex (вершины) и UiEdge (рёбра), которые наследуют графические элементы Qt и отвечают за отрисовку, цветовую индикацию и состояние рёбер.

Класс GraphWorker выполняет алгоритмы (BFS, DFS, Dijkstra, Floyd), управляет пошаговой визуализацией и связывает модель Graph с графическим представлением.

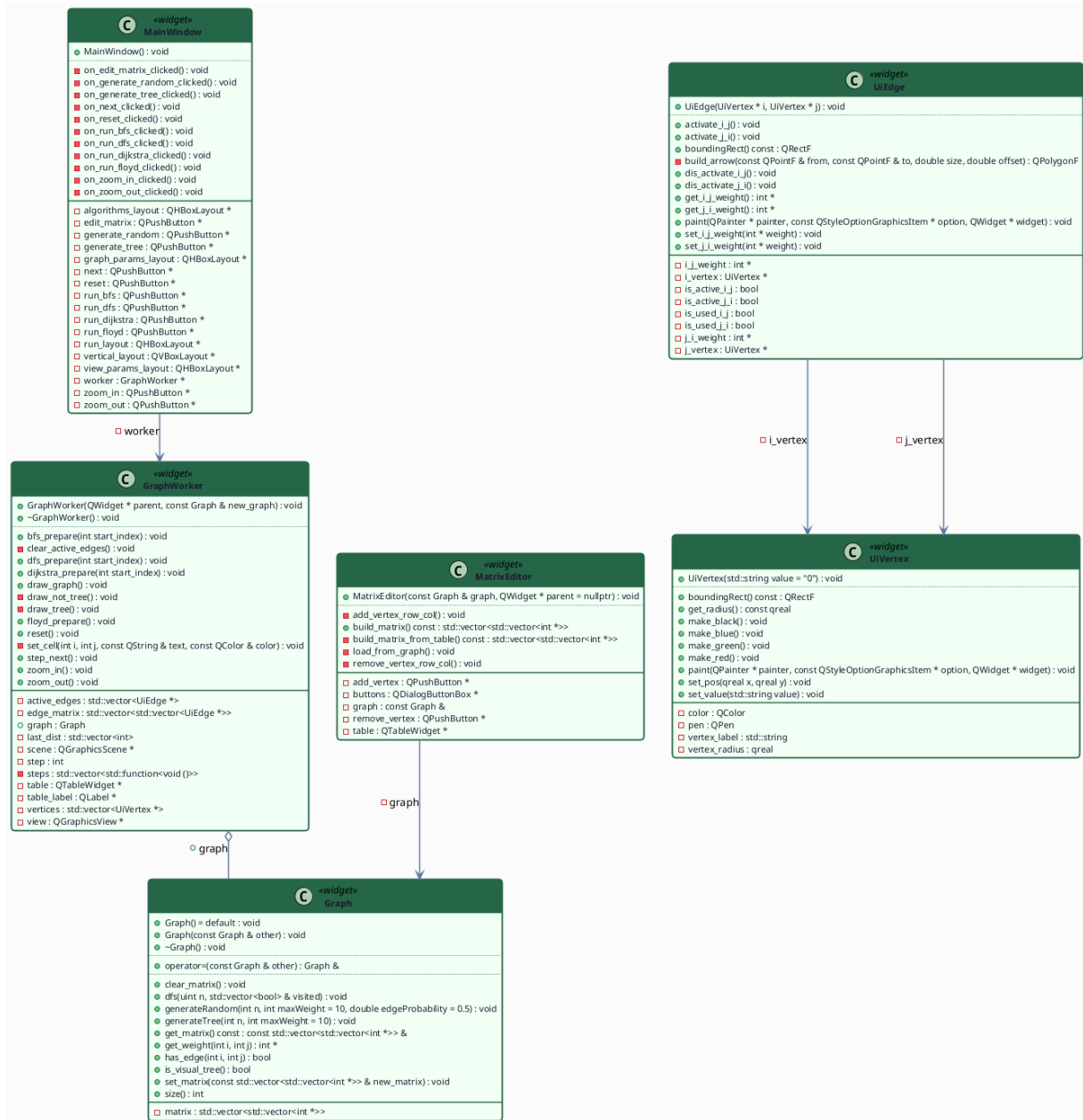
MatrixEditor отвечает за редактирование матрицы смежности через таблицу Qt и синхронизацию с Graph.

MainWindow выступает контроллером интерфейса, обрабатывает события кнопок и управляет запуском алгоритмов через GraphWorker.

Вариант:



UML-ДИАГРАММА



КОД UiVertex.h

```

//
// Created by localuser on 5/12/26.
//

#ifndef QTGRAPHVISUALIZATION_UIVERTEX_H
#define QTGRAPHVISUALIZATION_UIVERTEX_H
#include <QGraphicsScene>
#include <QGraphicsItem>

```

```

#include <QPainter>

class UiVertex : public QGraphicsItem {
    std::string vertex_label = "0";
    qreal vertex_radius = 30;

    QColor color = Qt::black;
    QPen pen = QPen(Qt::black, 3);
public:
    UiVertex(std::string value = "0");

    void set_pos(qreal x, qreal y);
    void set_value(std::string value);
    const qreal get_radius();

    void make_red();
    void make_green();
    void make_blue();
    void make_black();
    QRectF boundingRect() const override;

    void paint(QPainter *painter,
               const QStyleOptionGraphicsItem *option,
               QWidget *widget) override;
};

#endif //QTGRAPHVISUALIZATION_UIVERTEX_H

```

КОД UiVertex.cpp

```
//  
// Created by localuser on 5/12/26.  
//  
  
#include "UiVertex.h"  
  
UiVertex::UiVertex(std::string value) {  
    vertex_label = value;  
}  
  
QRectF UiVertex::boundingRect() const {  
    return QRectF(-vertex_radius, -vertex_radius,  
        vertex_radius * 2,  
        vertex_radius * 2);  
}  
  
void UiVertex::paint(QPainter *painter,  
    const QStyleOptionGraphicsItem *,  
    QWidget *) {  
  
    painter->setPen(pen);  
  
    painter->setBrush(Qt::white);  
    painter->drawEllipse(boundingRect());  
  
    painter->setPen(Qt::black);  
    painter->drawText(boundingRect(),  
        Qt::AlignCenter,  
        QString::fromStdString(vertex_label));  
}  
  
void UiVertex::set_pos(qreal x, qreal y) {  
    setPos(x, y);  
}  
  
void UiVertex::set_value(std::string value) {  
    vertex_label = value;  
    update();  
}  
  
const qreal UiVertex::get_radius() {  
    return vertex_radius;  
}
```

```
void UiVertex::make_red() {  
    pen = QPen(Qt::red, 5);  
    update();  
}  
void UiVertex::make_blue() {  
    pen = QPen(Qt::blue, 5);  
    update();  
}  
void UiVertex::make_green() {  
    pen = QPen(Qt::green, 3);  
    update();  
}  
  
void UiVertex::make_black() {  
    pen = QPen(Qt::black, 3);  
    update();  
}
```

Код UiEdge.h

```
//  
// Created by localuser on 5/12/26.  
//  
  
#ifndef QTGRAPHVISUALIZATION_UIEDGE_H  
#define QTGRAPHVISUALIZATION_UIEDGE_H  
#include <QGraphicsItem>  
#include "UiVertex.h"  
  
class UiEdge : public QGraphicsItem {  
    UiVertex *i_vertex;  
    UiVertex *j_vertex;  
    int* i_j_weight = nullptr;  
    int* j_i_weight = nullptr;  
    bool is_active_i_j = false;  
    bool is_active_j_i = false;  
    bool is_used_i_j = false;  
    bool is_used_j_i = false;  
    QPolygonF build_arrow(const QPointF &from, const QPointF &to, double size, double offset);  
  
public:  
    UiEdge(UiVertex *i, UiVertex *j);  
  
    QRectF boundingRect() const override;  
  
    void paint(QPainter *painter,  
               const QStyleOptionGraphicsItem *option,  
               QWidget *widget) override;  
    void set_i_j_weight(int* weight);  
    void set_j_i_weight(int* weight);  
  
    void activate_i_j();  
    void dis_activate_i_j();  
  
    void activate_j_i();  
    void dis_activate_j_i();  
  
    int* get_i_j_weight();  
    int* get_j_i_weight();  
};
```



```
#endif //QTGRAPHVISUALIZATION_UIEDGE_H
```

КОД UiEdge.cpp

```
//  
// Created by localuser on 5/12/26.  
//  
  
#include "UiEdge.h"  
  
#include "UiVertex.h"  
  
UiEdge::UiEdge(UiVertex *i, UiVertex *j) {  
    i_vertex = i;  
    j_vertex = j;  
}  
  
QRectF UiEdge::boundingRect() const {  
    auto aleft = std::min(i_vertex->pos().x(), j_vertex->pos().x()) - i_vertex->get_radius();  
    auto atop = std::min(i_vertex->pos().y(), j_vertex->pos().y()) - i_vertex->get_radius();  
    auto awidth = abs(i_vertex->pos().x() - j_vertex->pos().x()) + 2 * i_vertex->get_radius();  
    auto aheight = abs(i_vertex->pos().y() - j_vertex->pos().y()) + 2 * i_vertex->get_radius();  
  
    return QRectF(aleft, atop,  
                  awidth,  
                  aheight);  
}  
  
void UiEdge::paint(QPainter *painter,  
                  const QStyleOptionGraphicsItem *option,  
                  QWidget *widget) {  
    if (i_vertex->pos() == j_vertex->pos())  
        return;  
  
    painter->setPen(is_active_i_j || is_active_j_i ? QPen(Qt::red, 5) : QPen(Qt::black, 3));  
    painter->drawLine(i_vertex->pos(), j_vertex->pos());  
  
    if (i_j_weight != nullptr) {  
        QPolygonF arrow = build_arrow(i_vertex->pos(), j_vertex->pos(), i_vertex->get_radius() * 2,  
                                       i_vertex->get_radius());  
  
        painter->setPen(is_active_i_j ? QPen(Qt::red, 5) : QPen(Qt::black, 3));
```

```

painter->setBrush(QBrush(Qt::white));
painter->drawPolygon(arrow);

QPointF centroid = (arrow[0] + arrow[1] + arrow[2]) / 3.0;
painter->setPen(QPen(Qt::black));
QString text = QString::number(*this->i_j_weight);
if (*i_j_weight == INT_MAX)
    text = "+inf";
if (*i_j_weight == INT_MIN)
    text = "-inf";
painter->drawText(
    (int) (centroid.x() - 20),
    (int) (centroid.y() - 10),
    40, 20,
    Qt::AlignCenter,
    text
);
}

if (j_i_weight != nullptr) {
    QPolygonF arrow = build_arrow(j_vertex->pos(), i_vertex->pos(), i_vertex->get_radius() * 2,
                                   i_vertex->get_radius());

    painter->setPen(is_active_j_i ? QPen(Qt::red, 5) : QPen(Qt::black, 3));
    painter->setBrush(QBrush(Qt::white));
    painter->drawPolygon(arrow);

    QPointF centroid = (arrow[0] + arrow[1] + arrow[2]) / 3.0;
    painter->setPen(QPen(Qt::black));
    QString text = QString::number(*this->j_i_weight);
    if (*j_i_weight == INT_MAX)
        text = "+inf";
    if (*j_i_weight == INT_MIN)
        text = "-inf";
    painter->drawText(
        (int) (centroid.x() - 20),
        (int) (centroid.y() - 10),
        40, 20,
        Qt::AlignCenter,
        text
    );
}
}

```

```

QPolygonF UiEdge::build_arrow(const QPointF &from, const QPointF &to,

```

```

        double size, double offset) {
    QPointF dir = to - from;
    double len = std::sqrt(dir.x() * dir.x() + dir.y() * dir.y());
    if (len == 0) return QPolygonF();

    QPointF unit = dir / len;
    QPointF perp(-unit.y(), unit.x());

    double height = size * std::sqrt(3.0) / 2.0;

    QPointF tip = to - unit * offset;
    QPointF base1 = tip - unit * height + perp * (size / 2.0);
    QPointF base2 = tip - unit * height - perp * (size / 2.0);
    QPolygonF pol;
    pol.resize(3);
    pol[0] = tip;
    pol[1] = base1;
    pol[2] = base2;
    return pol;
}

void UiEdge::set_i_j_weight(int *weight) {
    i_j_weight = weight;
    update();
}

void UiEdge::set_j_i_weight(int *weight) {
    j_i_weight = weight;
    update();
}

void UiEdge::activate_i_j() {
    is_active_i_j = true;
    update();
}

void UiEdge::dis_activate_i_j() {
    is_active_i_j = false;
    update();
}

void UiEdge::activate_j_i() {
    is_active_j_i = true;
    update();
}

```

```

void UiEdge::dis_activate_j_i() {
    is_active_j_i = false;
    update();
}

int *UiEdge::get_i_j_weight() { return i_j_weight; }

int *UiEdge::get_j_i_weight() { return j_i_weight; }

```

КОД MatrixEditor.h

```

//
// Created by localuser on 5/13/26.
//

#ifndef QTGRAPHVISUALIZATION_MATRIXEDITOR_H
#define QTGRAPHVISUALIZATION_MATRIXEDITOR_H

#include <QDialog>
#include <QTableWidget>
#include <QDialogButtonBox>
#include <QPushButton>
#include "Graph.h"

class MatrixEditor : public QDialog {
    Q_OBJECT

    const Graph &graph;
    QTableWidget *table;
    QPushButton *add_vertex;
    QPushButton *remove_vertex;
    QDialogButtonBox *buttons;

    void load_from_graph();
    void add_vertex_row_col();
    void remove_vertex_row_col();
    std::vector<std::vector<int *> > build_matrix_from_table() const;

public:
    MatrixEditor(const Graph &graph, QWidget *parent = nullptr);
    std::vector<std::vector<int *> > build_matrix() const;
};

```

```
#endif //QTGRAPHVISUALIZATION_MATRIXEDITOR_H
```

КОД MainWindow.h

```
//  
// Created by localuser on 5/12/26.  
//  
  
#include "MainWindow.h"  
  
#include "UiEdge.h"  
  
MainWindow::MainWindow() : QWidget(nullptr){  
    vertical_layout = new QVBoxLayout(this);  
  
    Graph graph;  
    worker = new GraphWorker(this, graph);  
    worker->draw_graph();  
  
    graph_params_layout = new QHBoxLayout(nullptr);  
    edit_matrix = new QPushButton("Edit matrix", this);  
    generate_random = new QPushButton("Generate random graph", this);  
    generate_tree = new QPushButton("Generate tree", this);  
  
    graph_params_layout->addWidget(edit_matrix);  
    graph_params_layout->addWidget(generate_random);  
    graph_params_layout->addWidget(generate_tree);  
    vertical_layout->addLayout(graph_params_layout);  
  
    vertical_layout->addWidget(worker);  
    algorithms_layout = new QHBoxLayout;  
    vertical_layout->addLayout(algorithms_layout);  
  
    run_bfs = new QPushButton("BFS!", this);  
    run_dfs = new QPushButton("DFS!", this);  
    run_dijkstra = new QPushButton("Dijkstra!", this);  
    run_floyd = new QPushButton("Floyd!", this);  
  
    algorithms_layout->addWidget(run_bfs);  
    algorithms_layout->addWidget(run_dfs);  
    algorithms_layout->addWidget(run_dijkstra);  
    algorithms_layout->addWidget(run_floyd);
```

```

graph_params_layout = new QHBoxLayout;
vertical_layout->addLayout(graph_params_layout);

run_layout = new QHBoxLayout(nullptr);
reset = new QPushButton("Reset", this);
next = new QPushButton("Next step", this);
run_layout->addWidget(reset);
run_layout->addWidget(next);

vertical_layout->addLayout(run_layout);

zoom_in = new QPushButton("Zoom in", this);
zoom_out = new QPushButton("Zoom out", this);

graph_params_layout->addWidget(zoom_in);
graph_params_layout->addWidget(zoom_out);

connect(edit_matrix, &QPushButton::clicked, this, &MainWindow::on_edit_matrix_clicked);
connect(generate_random, &QPushButton::clicked, this,
&MainWindow::on_generate_random_clicked);
connect(generate_tree, &QPushButton::clicked, this,
&MainWindow::on_generate_tree_clicked);

connect(zoom_in, &QPushButton::clicked, this, &MainWindow::on_zoom_in_clicked);
connect(zoom_out, &QPushButton::clicked, this, &MainWindow::on_zoom_out_clicked);

connect(run_dfs, &QPushButton::clicked, this, &MainWindow::on_run_dfs_clicked);
connect(run_bfs, &QPushButton::clicked, this, &MainWindow::on_run_bfs_clicked);
connect(run_dijkstra, &QPushButton::clicked, this, &MainWindow::on_run_dijkstra_clicked);
connect(run_floyd, &QPushButton::clicked, this, &MainWindow::on_run_floyd_clicked);
connect(next, &QPushButton::clicked, this, &MainWindow::on_next_clicked);
connect(reset, &QPushButton::clicked, this, &MainWindow::on_reset_clicked);
}

void MainWindow::on_edit_matrix_clicked() {
    MatrixEditor editor(worker->graph, this);
    if (editor.exec() == QDialog::Accepted) {
        worker->graph.set_matrix(editor.build_matrix());
        worker->draw_graph();
    }
}

void MainWindow::on_zoom_in_clicked() {
    worker->zoom_in();
}

```

```

void MainWindow::on_zoom_out_clicked() {
    worker->zoom_out();
}

void MainWindow::on_generate_tree_clicked() {
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 50);
    spin->setValue(1);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateTree(spin->value(), 10);
    worker->draw_graph();
}

void MainWindow::on_generate_random_clicked() {
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 50);
    spin->setValue(1);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
}

```

```

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateRandom(spin->value(), 10, 0.3);
    worker->draw_graph();
}

void MainWindow::on_run_dfs_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

    QDialog dialog(this);
    dialog.setWindowTitle("DFS start vertex");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, n);
    spin->setValue(1);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->reset();
    worker->dfs_prepare(spin->value() - 1);
}

void MainWindow::on_run_bfs_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

    QDialog dialog(this);
    dialog.setWindowTitle("BFS start vertex");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, n);
    spin->setValue(1);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,

```



```

&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->reset();
    worker->bfs_prepare(spin->value() - 1);
}

void MainWindow::on_run_dijkstra_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

    QDialog dialog(this);
    dialog.setWindowTitle("Dijkstra start vertex");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, n);
    spin->setValue(1);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->reset();
    worker->dijkstra_prepare(spin->value() - 1);
}

void MainWindow::on_run_floyd_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

    worker->reset();

```

```
    worker->floyd_prepare0;  
}
```

```
void MainWindow::on_next_clicked() {  
    worker->step_next();  
}
```

```
void MainWindow::on_reset_clicked() {  
    worker->reset();  
}
```

КОД MainWindow.cpp

```
//  
// Created by localuser on 5/12/26.  
//
```

```
#include "MainWindow.h"
```

```
#include "UiEdge.h"
```

```
MainWindow::MainWindow() : QWidget(nullptr){  
    vertical_layout = new QVBoxLayout(this);  
  
    Graph graph;  
    worker = new GraphWorker(this, graph);  
    worker->draw_graph();  
  
    graph_params_layout = new QHBoxLayout(nullptr);  
    edit_matrix = new QPushButton("Edit matrix", this);  
    generate_random = new QPushButton("Generate random graph", this);  
    generate_tree = new QPushButton("Generate tree", this);  
  
    graph_params_layout->addWidget(edit_matrix);  
    graph_params_layout->addWidget(generate_random);  
    graph_params_layout->addWidget(generate_tree);  
    vertical_layout->addLayout(graph_params_layout);  
  
    vertical_layout->addWidget(worker);  
    algorithms_layout = new QHBoxLayout;  
    vertical_layout->addLayout(algorithms_layout);  
  
    run_bfs = new QPushButton("BFS!", this);  
    run_dfs = new QPushButton("DFS!", this);
```

```

run_dijkstra = new QPushButton("Dijkstra!", this);
run_floyd = new QPushButton("Floyd!", this);

algorithms_layout->addWidget(run_bfs);
algorithms_layout->addWidget(run_dfs);
algorithms_layout->addWidget(run_dijkstra);
algorithms_layout->addWidget(run_floyd);

graph_params_layout = new QHBoxLayout;
vertical_layout->addLayout(graph_params_layout);

run_layout = new QHBoxLayout(nullptr);
reset = new QPushButton("Reset", this);
next = new QPushButton("Next step", this);
run_layout->addWidget(reset);
run_layout->addWidget(next);

vertical_layout->addLayout(run_layout);

zoom_in = new QPushButton("Zoom in", this);
zoom_out = new QPushButton("Zoom out", this);

graph_params_layout->addWidget(zoom_in);
graph_params_layout->addWidget(zoom_out);

connect(edit_matrix, &QPushButton::clicked, this, &MainWindow::on_edit_matrix_clicked);
connect(generate_random, &QPushButton::clicked, this,
&MainWindow::on_generate_random_clicked);
connect(generate_tree, &QPushButton::clicked, this,
&MainWindow::on_generate_tree_clicked);

connect(zoom_in, &QPushButton::clicked, this, &MainWindow::on_zoom_in_clicked);
connect(zoom_out, &QPushButton::clicked, this, &MainWindow::on_zoom_out_clicked);

connect(run_dfs, &QPushButton::clicked, this, &MainWindow::on_run_dfs_clicked);
connect(run_bfs, &QPushButton::clicked, this, &MainWindow::on_run_bfs_clicked);
connect(run_dijkstra, &QPushButton::clicked, this, &MainWindow::on_run_dijkstra_clicked);
connect(run_floyd, &QPushButton::clicked, this, &MainWindow::on_run_floyd_clicked);
connect(next, &QPushButton::clicked, this, &MainWindow::on_next_clicked);
connect(reset, &QPushButton::clicked, this, &MainWindow::on_reset_clicked);
}

void MainWindow::on_edit_matrix_clicked() {
    MatrixEditor editor(worker->graph, this);
    if (editor.exec() == QDialog::Accepted) {

```

```

        worker->graph.set_matrix(editor.build_matrix());
        worker->draw_graph();
    }
}

void MainWindow::on_zoom_in_clicked() {
    worker->zoom_in();
}

void MainWindow::on_zoom_out_clicked() {
    worker->zoom_out();
}

void MainWindow::on_generate_tree_clicked() {
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 50);
    spin->setValue(1);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateTree(spin->value(), 10);
    worker->draw_graph();
}

void MainWindow::on_generate_random_clicked() {
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 50);
    spin->setValue(1);

```

```

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateRandom(spin->value(), 10, 0.3);
    worker->draw_graph();
}

```

```

void MainWindow::on_run_dfs_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

```

```

    QDialog dialog(this);
    dialog.setWindowTitle("DFS start vertex");

```

```

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, n);
    spin->setValue(1);

```

```

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->reset();
    worker->dfs_prepare(spin->value() - 1);
}

```

```

void MainWindow::on_run_bfs_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

```

```

    QDialog dialog(this);

```

```

dialog.setWindowTitle("BFS start vertex");

auto *layout = new QVBoxLayout(&dialog);
auto *spin = new QSpinBox(&dialog);
spin->setRange(1, n);
spin->setValue(1);

auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
layout->addWidget(spin);
layout->addWidget(buttons);

connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

if (dialog.exec() != QDialog::Accepted) return;

worker->reset();
worker->bfs_prepare(spin->value() - 1);
}

void MainWindow::on_run_dijkstra_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

    QDialog dialog(this);
    dialog.setWindowTitle("Dijkstra start vertex");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, n);
    spin->setValue(1);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->reset();
    worker->dijkstra_prepare(spin->value() - 1);
}

```

```
}
```

```
void MainWindow::on_run_floyd_clicked() {  
    int n = worker->graph.size();  
    if (n == 0) return;  
  
    worker->reset();  
    worker->floyd_prepare();  
}
```

```
void MainWindow::on_next_clicked() {  
    worker->step_next();  
}
```

```
void MainWindow::on_reset_clicked() {  
    worker->reset();  
}
```

КОД main.cpp

```
#include <QApplication>  
#include <QPushButton>  
#include "MainWindow.h"  
  
int main(int argc, char* argv[]) {  
    QApplication a(argc, argv);  
    MainWindow window;  
    window.show();  
    return QApplication::exec();  
}
```

КОД GraphWorker.h

```
//  
// Created by localuser on 5/12/26.  
//  
  
#ifndef QTGRAPHVISUALIZATION_GRAPHWORKER_H  
#define QTGRAPHVISUALIZATION_GRAPHWORKER_H  
#include <QGraphicsView>  
#include <QTableWidget>  
#include <QLabel>  
#include <QWidget>
```

```

#include <vector>
#include "UiVertex.h"
#include "UiEdge.h"
#include "Graph.h"

class GraphWorker : public QWidget {
    Q_OBJECT
    std::vector<UiVertex*> vertices;
    std::vector<std::vector<UiEdge*> > edge_matrix;
    std::vector<std::function<void()>> steps;
    int step = 0;
    std::vector<int> last_dist;
    std::vector<UiEdge*> active_edges;

    QGraphicsScene *scene;
    QGraphicsView *view;

    QWidget* table;
    QLabel* table_label;

    void draw_tree();
    void draw_not_tree();
    void clear_active_edges();
    void set_cell(int i, int j, const QString& text, const QColor& color);
public:
    GraphWorker(QWidget* parent, const Graph& new_graph);
    ~GraphWorker();

    Graph graph;
    void draw_graph();
    void zoom_in();
    void zoom_out();
    void dfs_prepare(int start_index);
    void step_next();
    void reset();
    void bfs_prepare(int start_index);
    void dijkstra_prepare(int start_index);

    void floyd_prepare();
};

#endif //QTGRAPHVISUALIZATION_GRAPHWORKER_H

```


КОД GraphWorker.cpp

```
//  
// Created by localuser on 5/12/26.  
//  
  
#include "GraphWorker.h"  
  
#include <QGraphicsView>  
#include <QVBoxLayout>  
#include <cmath>  
  
void assign_tree_positions(int v,  
                           int depth,  
                           const std::vector<std::vector<int> > &children,  
                           std::vector<double> &x_pos,  
                           std::vector<double> &y_pos,  
                           int &leaf_counter,  
                           double h_step,  
                           double v_step) {  
    y_pos[v] = depth * v_step + 60.0;  
    if (children[v].empty()) {  
        x_pos[v] = leaf_counter * h_step + 60.0;  
        leaf_counter++;  
        return;  
    }  
  
    for (int c: children[v])  
        assign_tree_positions(c, depth + 1, children, x_pos, y_pos, leaf_counter, h_step, v_step);  
  
    x_pos[v] = (x_pos[children[v].front()] + x_pos[children[v].back()]) / 2.0;  
}  
  
void GraphWorker::clear_active_edges() {  
    for (auto *edge: active_edges) {  
        if (!edge) continue;  
        edge->dis_activate_i_j();  
        edge->dis_activate_j_i();  
        edge->update();  
    }  
    active_edges.clear();  
}  
  
void GraphWorker::draw_tree() {
```

```

scene->clear();
vertices.clear();
edge_matrix.clear();

int n = graph.size();
if (n == 0) return;

std::vector<int> in_degree(n, 0);
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        if (i != j && graph.has_edge(i, j))
            in_degree[j]++;

int root = 0;
for (int i = 0; i < n; i++)
    if (in_degree[i] == 0) {
        root = i;
        break;
    }

std::vector<std::vector<int> > children(n);
std::vector<bool> used(n, false);
std::queue<int> q;
q.push(root);
used[root] = true;

while (!q.empty()) {
    int cur = q.front();
    q.pop();
    for (int next = 0; next < n; next++) {
        if (!used[next] && graph.has_edge(cur, next)) {
            used[next] = true;
            children[cur].push_back(next);
            q.push(next);
        }
    }
}

const double H_STEP = 200.0;
const double V_STEP = 200.0;

std::vector<double> x_pos(n, 0.0);
std::vector<double> y_pos(n, 0.0);

int leaf_counter = 0;

```

```

assign_tree_positions(root, 0, children, x_pos, y_pos, leaf_counter, H_STEP, V_STEP);

double min_x = x_pos[0];
double max_x = x_pos[0];
double min_y = y_pos[0];
double max_y = y_pos[0];
for (int i = 1; i < n; i++) {
    min_x = std::min(min_x, x_pos[i]);
    max_x = std::max(max_x, x_pos[i]);
    min_y = std::min(min_y, y_pos[i]);
    max_y = std::max(max_y, y_pos[i]);
}

QPointF center = view->mapToScene(view->viewport()->rect().center());
double shift_x = center.x() - (min_x + max_x) / 2.0;
double shift_y = center.y() - (min_y + max_y) / 2.0;

for (int i = 0; i < n; i++) {
    auto *v = new UiVertex(std::to_string(i + 1));
    v->set_pos(x_pos[i] + shift_x, y_pos[i] + shift_y);
    vertices.push_back(v);
}

edge_matrix.assign(n, std::vector<UiEdge *>(n, nullptr));

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (graph.has_edge(i, j) || graph.has_edge(j, i)) {
            auto *e = new UiEdge(vertices[i], vertices[j]);
            if (graph.has_edge(i, j)) {
                e->set_i_j_weight(graph.get_weight(i, j));
                edge_matrix[i][j] = e;
            }
            if (graph.has_edge(j, i)) {
                e->set_j_i_weight(graph.get_weight(j, i));
                edge_matrix[j][i] = e;
            }
        }
    }
}

for (auto v: edge_matrix)
    for (auto ui_edge: v)
        if (ui_edge != nullptr)
            scene->addItem(ui_edge);

```

```

    for (auto vertex: vertices) scene->addItem(vertex);
}

void GraphWorker::draw_not_tree() {
    scene->clear();
    vertices.clear();
    edge_matrix.clear();

    int n = graph.size();
    if (n == 0) return;

    const double radius = 500.0;
    const double step = 2.0 * M_PI / n;
    QPointF center = view->mapToScene(view->viewport()->rect().center());

    for (int i = 0; i < n; i++) {
        double angle = step * i;
        double x = center.x() + radius * std::cos(angle);
        double y = center.y() + radius * std::sin(angle);
        auto *v = new UiVertex(std::to_string(i + 1));
        v->set_pos(x, y);
        vertices.push_back(v);
    }

    edge_matrix.assign(n, std::vector<UiEdge *>(n, nullptr));

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (graph.has_edge(i, j) || graph.has_edge(j, i)) {
                auto *e = new UiEdge(vertices[i], vertices[j]);
                if (graph.has_edge(i, j)) {
                    e->set_i_j_weight(graph.get_weight(i, j));
                    edge_matrix[i][j] = e;
                }
                if (graph.has_edge(j, i)) {
                    e->set_j_i_weight(graph.get_weight(j, i));
                    edge_matrix[j][i] = e;
                }
            }
        }
    }
}

for (auto v: edge_matrix)
    for (auto ui_edge: v)
        if (ui_edge != nullptr)

```

```

        scene->addItem(ui_edge);
    for (auto vertex: vertices) scene->addItem(vertex);
}

```

```

GraphWorker::GraphWorker(QWidget *parent, const Graph &new_graph) : QWidget(parent) {
    table = new QTableWidget(this);
    table->hide();
    table->setMaximumHeight(200);
    table_label = new QLabel(this);
    table_label->hide();

    scene = new QGraphicsScene(this);
    view = new QGraphicsView(scene, this);
    graph = new_graph;

    view->setRenderHint(QPainter::Antialiasing);
    view->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    view->setTransformationAnchor(QGraphicsView::AnchorUnderMouse);
    view->setInteractive(true);
    view->setDragMode(QGraphicsView::ScrollHandDrag);

    QVBoxLayout *layout = new QVBoxLayout(this);
    layout->setContentsMargins(0, 0, 0, 0);
    layout->addWidget(view);
    layout->addWidget(table_label);
    layout->addWidget(table);
    setLayout(layout);

    setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
}

```

```

GraphWorker::~GraphWorker() {
    for (auto *v: vertices) delete v;
    for (auto v: edge_matrix)
        for (auto ui_edge: v)
            if (ui_edge != nullptr)
                delete ui_edge;
}

```

```

void GraphWorker::zoom_in() {
    view->setTransformationAnchor(QGraphicsView::AnchorViewCenter);
    view->scale(1.2, 1.2);
}

```

```

void GraphWorker::zoom_out() {

```

```

view->setTransformationAnchor(QGraphicsView::AnchorViewCenter);
view->scale(1.0 / 1.2, 1.0 / 1.2);
}

```

```

void GraphWorker::draw_graph() {
    if (graph.is_visual_tree()) {
        draw_tree();
        return;
    }
    draw_not_tree();
}

```

```

void GraphWorker::dfs_prepare(int start_index) {
    steps.clear();
    step = 0;
    int n = graph.size();

    std::vector<bool> used(n, false);
    std::function<void(int, int)> dfs = [&](int node, int prev) {
        used[node] = true;

```

```

        steps.push_back([this, node, prev]() {
            vertices[prev]->make_blue();
            vertices[node]->make_red();
            auto edge = edge_matrix[prev][node];
            if (edge) {
                if (node < prev)
                    edge->dis_activate_j_i();
                else if (node > prev)
                    edge->dis_activate_i_j();
            }
        });

```

```

        for (int j = 0; j < n; j++) {
            if (graph.has_edge(node, j) && !used[j]) {
                steps.push_back([this, node, j]() {
                    UiEdge *edge = edge_matrix[node][j];
                    if (node < j)
                        edge->activate_i_j();
                    else
                        edge->activate_j_i();
                });

                dfs(j, node);

```

```

        steps.push_back([this, node, j]) {
            vertices[node]->make_red();
            vertices[j]->make_blue();
        });
    }
}

};

dfs(start_index, start_index);
}

void GraphWorker::step_next() {
    if (step >= static_cast<int>(steps.size())) return;
    steps[step]();
    step++;
    qDebug() << step;
}

void GraphWorker::reset() {
    steps.clear();
    step = 0;
    clear_active_edges();

    for (int i = 0; i < static_cast<int>(vertices.size()); i++) {
        vertices[i]->make_black();
        vertices[i]->set_value(std::to_string(i + 1));
        vertices[i]->update();
    }
    for (auto v: edge_matrix)
        for (auto ui_edge: v)
            if (ui_edge != nullptr) {
                ui_edge->dis_activate_i_j();
                ui_edge->dis_activate_j_i();
                ui_edge->update();
            }
    table->hide();
    table_label->hide();
}

void GraphWorker::bfs_prepare(int start_index) {
    steps.clear();
    step = 0;
    int n = graph.size();
    std::vector<bool> used(n, false);

```

```

std::queue<int> q;

used[start_index] = true;
q.push(start_index);

while (!q.empty()) {
    int node = q.front();
    q.pop();

    for (int j = 0; j < n; j++) {
        if (graph.has_edge(node, j) && !used[j]) {
            used[j] = true;
            q.push(j);
            steps.push_back([this, node]() {
                vertices[node]->make_red();
            });
            steps.push_back([this, node, j]() {
                auto edge = edge_matrix[node][j];
                if (edge)
                    node < j ? edge->activate_i_j() : edge->activate_j_i();
            });

            steps.push_back([this, node, j]() {
                vertices[node]->make_blue();
                auto edge = edge_matrix[node][j];
                if (edge)
                    node < j ? edge->dis_activate_i_j() : edge->dis_activate_j_i();
                vertices[j]->make_red();
            });
            steps.push_back([this, j]() {
                vertices[j]->make_blue();
            });
        }
    }
}

```

```

void GraphWorker::dijkstra_prepare(int start_index) {
    steps.clear();
    step = 0;
    int n = graph.size();
    std::vector<bool> used(n, false);
    std::vector<int> d(n, INT_MAX);
    d[start_index] = 0;
}

```



```

for (int i = 0; i < n; i++)
    vertices[i]->set_value(std::to_string(i + 1) + "\n+inf");
vertices[start_index]->set_value(std::to_string(start_index + 1) + "\n0");

for (int i = 0; i < n; i++) {
    int v = -1;
    for (int j = 0; j < n; j++)
        if (!used[j] && (v == -1 || d[j] < d[v]))
            v = j;
    if (v == -1 || d[v] == INT_MAX) break;
    used[v] = true;

    steps.push_back([this, v]() {
        vertices[v]->make_red();
    });

    for (int e = 0; e < n; e++) {
        auto weight = graph.get_matrix0[v][e];
        if (weight == nullptr) continue;
        if (d[v] + *weight >= d[e]) continue;

        d[e] = d[v] + *weight;
        int new_dist = d[e];
        std::string new_val = std::to_string(e + 1) + "\n" + std::to_string(new_dist);

        steps.push_back([this, v, e, new_val]() {
            auto *edge = edge_matrix[v][e];
            if (edge)
                v < e ? edge->activate_i_j() : edge->activate_j_i();
            vertices[e]->make_green();
            vertices[e]->set_value(new_val);
        });

        steps.push_back([this, v, e]() {
            auto *edge = edge_matrix[v][e];
            if (edge)
                v < e ? edge->dis_activate_i_j() : edge->dis_activate_j_i();
            vertices[e]->make_black();
        });
    }

    steps.push_back([this, v]() {
        vertices[v]->make_blue();
    });
}

```

```

    }
}

void GraphWorker::set_cell(int i, int j, const QString &text, const QColor &color) {
    auto *item = table->item(i, j);
    if (!item) {
        item = new QTableWidgetItem();
        table->setItem(i, j, item);
    }
    item->setText(text);
    item->setBackground(color);
    item->setTextAlignment(Qt::AlignCenter);
}

void GraphWorker::floyd_prepare() {
    steps.clear();
    step = 0;
    int n = graph.size();

    table->show();
    table_label->show();
    table->setRowCount(n);
    table->setColumnCount(n);

    for (int i = 0; i < n; i++) {
        table->setHorizontalHeaderItem(i, new QTableWidgetItem(QString::number(i + 1)));
        table->setVerticalHeaderItem(i, new QTableWidgetItem(QString::number(i + 1)));
        table->setColumnWidth(i, 50);
        table->setRowHeight(i, 30);
    }

    std::vector<std::vector<int>> > d(n, std::vector<int>(n, INT_MAX));
    for (int i = 0; i < n; i++) d[i][i] = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (graph.get_matrix()[i][j] != nullptr)
                d[i][j] = *graph.get_matrix()[i][j];

    table_label->setText("Initial stage");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            QString val = d[i][j] == INT_MAX ? "+inf" : QString::number(d[i][j]);
            set_cell(i, j, val, Qt::white);
        }
}

```

```

for (int k = 0; k < n; k++) {
    steps.push_back([this, k, d, n]() {
        table_label->setText(QString("Throw: %1").arg(k + 1));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                QString val = d[i][j] == INT_MAX ? "+inf" : QString::number(d[i][j]);
                QColor color = (i == k || j == k) ? QColor(200, 200, 255) : Qt::white;
                set_cell(i, j, val, color);
            }
        for (auto *v: vertices) v->make_black();
        vertices[k]->make_blue();
    });

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (d[i][k] == INT_MAX || d[k][j] == INT_MAX) continue;
            if (d[i][k] + d[k][j] >= d[i][j]) continue;

            d[i][j] = d[i][k] + d[k][j];
            int new_dist = d[i][j];

            steps.push_back([this, i, j, k, d, n, new_dist]() {
                table_label->setText(QString("d[%1][%2] throw %3 = %4")
                    .arg(i + 1).arg(j + 1).arg(k + 1).arg(new_dist));

                for (int a = 0; a < n; a++)
                    for (int b = 0; b < n; b++) {
                        QString val = d[a][b] == INT_MAX ? "+inf" : QString::number(d[a][b]);
                        QColor color = (a == k || b == k) ? QColor(200, 200, 255) : Qt::white;
                        set_cell(a, b, val, color);
                    }

                for (int b = 0; b < n; b++)
                    table->item(i, b)->setBackground(QColor(255, 180, 180));
                for (int a = 0; a < n; a++)
                    table->item(a, j)->setBackground(QColor(255, 255, 180));
                for (int a = 0; a < n; a++) {
                    table->item(a, k)->setBackground(QColor(200, 200, 255));
                    table->item(k, a)->setBackground(QColor(200, 200, 255));
                }
                set_cell(i, j, QString::number(new_dist), QColor(180, 255, 180));

                for (auto *v: vertices) v->make_black();
                vertices[k]->make_blue();
                vertices[i]->make_red();
            });
        }
    }
}

```

```

        vertices[j]->make_green();
    });
}
}
}

steps.push_back([this, d, n]() {
    table_label->setText("The end");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            QString val = d[i][j] == INT_MAX ? "+inf" : QString::number(d[i][j]);
            set_cell(i, j, val, QColor(220, 255, 220));
        }
    for (auto *v: vertices) v->make_black();
});
}

```

КОД Graph.h

```

//
// Created by localuser on 5/13/26.
//

#include "MatrixEditor.h"

#include <QHeaderView>
#include <QVBoxLayout>
#include <QHBoxLayout>

MatrixEditor::MatrixEditor(const Graph &graph, QWidget *parent)
    : QDialog(parent), graph(graph) {
    setWindowTitle("Edit matrix");

    table = new QTableWidgetItem(this);
    table->setEditTriggers(QAbstractItemView::DoubleClicked
        | QAbstractItemView::SelectedClicked
        | QAbstractItemView::EditKeyPressed);
    table->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);
    table->verticalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    add_vertex = new QPushButton("Add vertex", this);
    remove_vertex = new QPushButton("Remove vertex", this);
    buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);

    auto *buttons_layout = new QHBoxLayout;

```

```

buttons_layout->addWidget(add_vertex);
buttons_layout->addWidget(remove_vertex);
buttons_layout->addWidget(buttons);

auto *main_layout = new QVBoxLayout(this);
main_layout->addWidget(table);
main_layout->addLayout(buttons_layout);
setLayout(main_layout);

load_from_graph();

connect(add_vertex, &QPushButton::clicked, this, &MatrixEditor::add_vertex_row_col);
connect(remove_vertex, &QPushButton::clicked, this,
&MatrixEditor::remove_vertex_row_col);
connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
}

void MatrixEditor::load_from_graph() {
    const auto &matrix = graph.get_matrix();
    int n = matrix.size();

    table->setRowCount(n);
    table->setColumnCount(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == nullptr) {
                table->setItem(i, j, new QTableWidgetItem(""));
            } else {
                table->setItem(i, j, new QTableWidgetItem(QString::number(*matrix[i][j])));
            }
        }
    }
}

std::vector<std::vector<int *>> MatrixEditor::build_matrix_from_table() const {
    int n = table->rowCount();
    std::vector<std::vector<int *>> matrix(n, std::vector<int *>(n, nullptr));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            QTableWidgetItem *item = table->item(i, j);
            if (!item) continue;
            QString text = item->text().trimmed();

```

```

        if (text.isEmpty()) continue;
        bool ok = false;
        int value = text.toInt(&ok);
        if (!ok) continue;
        matrix[i][j] = new int(value);
    }
}

return matrix;
}

void MatrixEditor::add_vertex_row_col() {
    int n = table->rowCount();
    table->setRowCount(n + 1);
    table->setColumnCount(n + 1);

    for (int i = 0; i < n + 1; i++) {
        for (int j = 0; j < n + 1; j++) {
            if (table->item(i, j)) continue;
            table->setItem(i, j, new QTableWidgetItem(""));
        }
    }
}

void MatrixEditor::remove_vertex_row_col() {
    int n = table->rowCount();
    table->setRowCount(n - 1);
    table->setColumnCount(n - 1);

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1; j++) {
            if (table->item(i, j)) continue;
            table->setItem(i, j, new QTableWidgetItem(""));
        }
    }
}

std::vector<std::vector<int*>> MatrixEditor::build_matrix() const {
    return build_matrix_from_table();
}

```

КОД MatrixEditor.cpp

//
 // Created by localuser on 5/12/26.

```
//

#ifdef QTGRAPHVISUALIZATION_GRAPH_H
#define QTGRAPHVISUALIZATION_GRAPH_H
#include <cstdlib>
#include <queue>
#include <vector>

class Graph {
    std::vector<std::vector<int *> > matrix;

public:
    Graph() = default;
    Graph(const Graph &other);
    ~Graph();
    Graph &operator=(const Graph &other);
    void clear_matrix();
    const std::vector<std::vector<int *> > &get_matrix() const;
    void set_matrix(const std::vector<std::vector<int *> > &new_matrix);

    bool is_visual_tree();

    void generateRandom(int n, int maxWeight = 10, double edgeProbability = 0.5);
    void generateTree(int n, int maxWeight = 10);

    int size();
    bool has_edge(int i, int j);
    int *get_weight(int i, int j);

    void dfs(uint n, std::vector<bool>& visited);
};

#endif //QTGRAPHVISUALIZATION_GRAPH_H
```

КОД Graph.cpp

```
#include "Graph.h"

#include <random>

static std::mt19937 rng(std::random_device{}());

Graph::Graph(const Graph &other) {
```

```

    set_matrix(other.matrix);
}

Graph::~~Graph() {
    clear_matrix();
}

void Graph::clear_matrix() {
    for (int i = 0; i < matrix.size(); i++)
        for (int j = 0; j < matrix[i].size(); j++)
            if (matrix[i][j] != nullptr)
                delete matrix[i][j];

    matrix.clear();
}

Graph &Graph::operator=(const Graph &other) {
    if (this == &other)
        return *this;

    set_matrix(other.matrix);

    return *this;
}

const std::vector<std::vector<int *> > &Graph::get_matrix() const {
    return matrix;
}

void Graph::set_matrix(const std::vector<std::vector<int *> > &new_matrix) {
    clear_matrix();

    matrix.resize(new_matrix.size());
    for (int i = 0; i < new_matrix.size(); i++)
        for (int j = 0; j < new_matrix[i].size(); j++)
            if (new_matrix[i][j] == nullptr)
                matrix[i].push_back(nullptr);
            else
                matrix[i].push_back(new int(*new_matrix[i][j]));
}

bool Graph::is_visual_tree() {
    int n = matrix.size();
    if (n == 0) return false;

```



```

int edgeCount = 0;
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        if (matrix[i][j] != nullptr || matrix[j][i] != nullptr)
            edgeCount++;

if (edgeCount != n - 1) return false;

std::vector<bool> visited(n, false);
std::queue<int> q;
q.push(0);
visited[0] = true;
int visitedCount = 1;

while (!q.empty()) {
    int cur = q.front();
    q.pop();

    for (int next = 0; next < n; next++) {
        if (!visited[next] &&
            (matrix[cur][next] != nullptr || matrix[next][cur] != nullptr)) {
            visited[next] = true;
            visitedCount++;
            q.push(next);
        }
    }
}

return visitedCount == n;
}

void Graph::generateRandom(int n, int maxWeight, double edgeProbability) {
    for (auto &row: matrix)
        for (auto *p: row)
            delete p;

    matrix.assign(n, std::vector<int*>(n, nullptr));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            std::bernoulli_distribution edge_dist(edgeProbability);
            if (edge_dist(rng)) {
                std::uniform_int_distribution<int> weight_dist(1, maxWeight);
                matrix[i][j] = new int(weight_dist(rng));
            }
        }
    }
}

```

```

    }
}
}
}

```

```

void Graph::generateTree(int n, int maxWeight) {
    for (auto &row: matrix)
        for (auto *p: row)
            delete p;

    matrix.assign(n, std::vector<int *>(n, nullptr));

    std::vector<int> perm(n);
    for (int i = 0; i < n; i++) perm[i] = i;
    for (int i = n - 1; i > 0; i--) {
        std::uniform_int_distribution<int> idx_dist(0, i);
        int j = idx_dist(rng);
        std::swap(perm[i], perm[j]);
    }

    for (int i = 1; i < n; i++) {
        int parent = perm[std::rand() % i];
        int child = perm[i];
        std::uniform_int_distribution<int> weight_dist(1, maxWeight);
        matrix[parent][child] = new int(weight_dist(rng));
    }
}

int Graph::size() {
    return matrix.size();
}

bool Graph::has_edge(int i, int j) {
    return matrix[i][j];
}

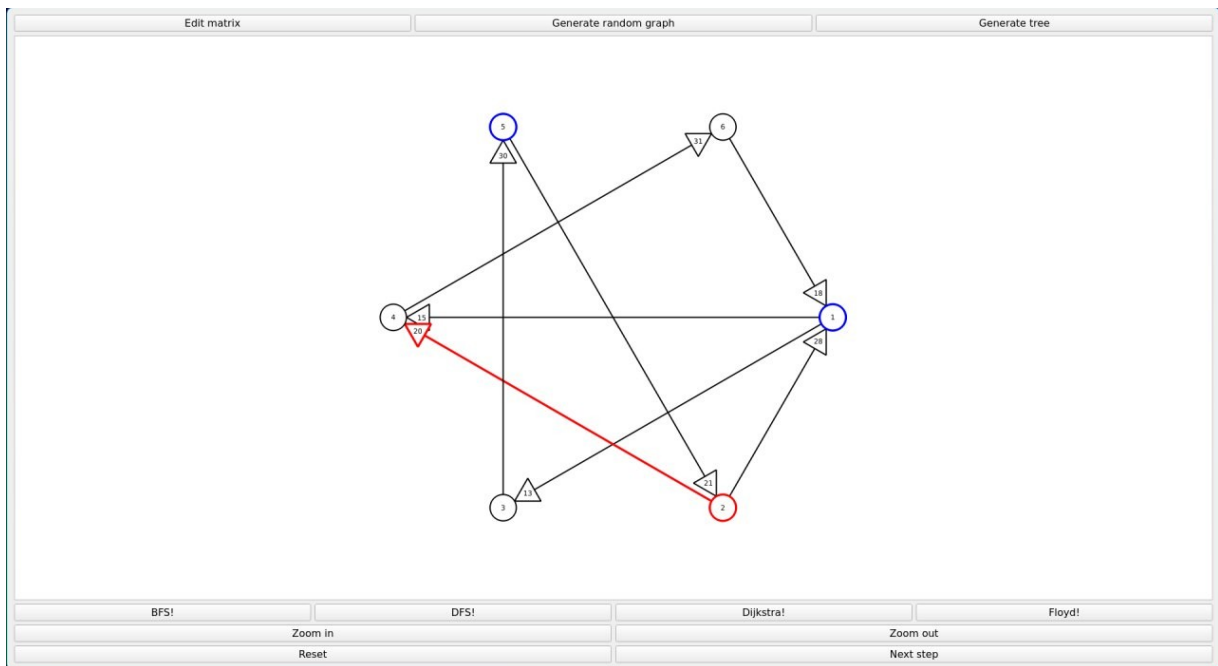
int *Graph::get_weight(int i, int j) {
    return matrix[i][j];
}

void Graph::dfs(uint n, std::vector<bool> &visited) {
    visited[n] = true;
    for (auto& i : matrix[n]) {
        if (i != nullptr && !visited[*i]) {
            dfs(*i, visited);
        }
    }
}

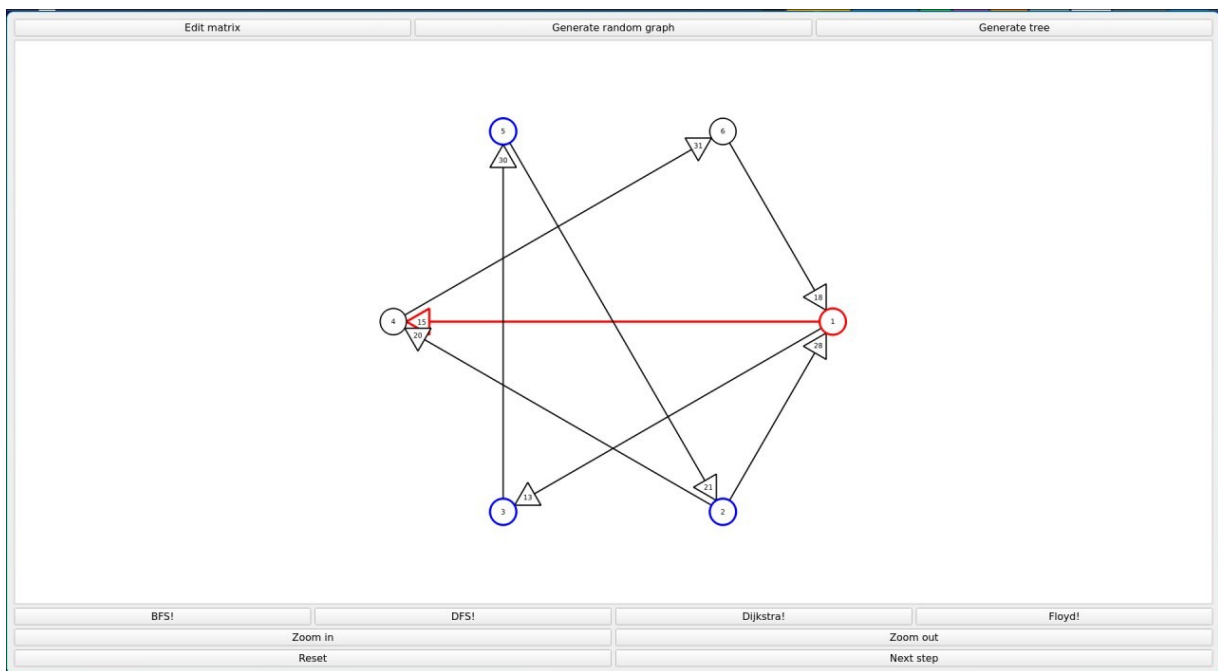
```

}
}
}

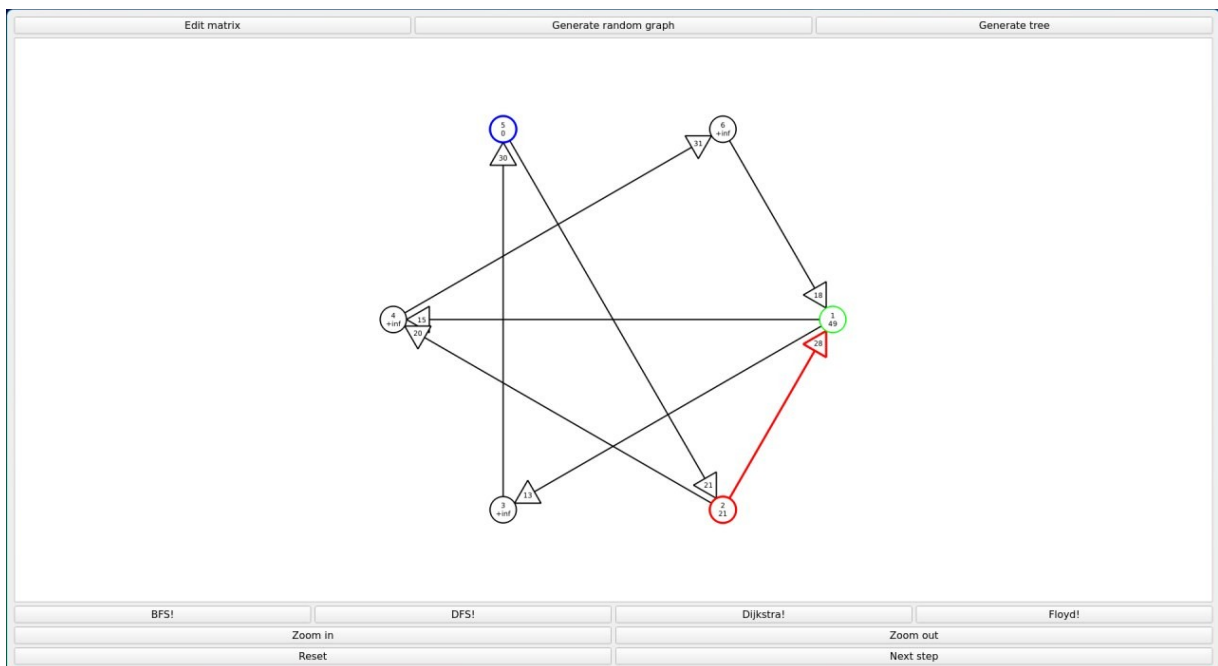
РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ



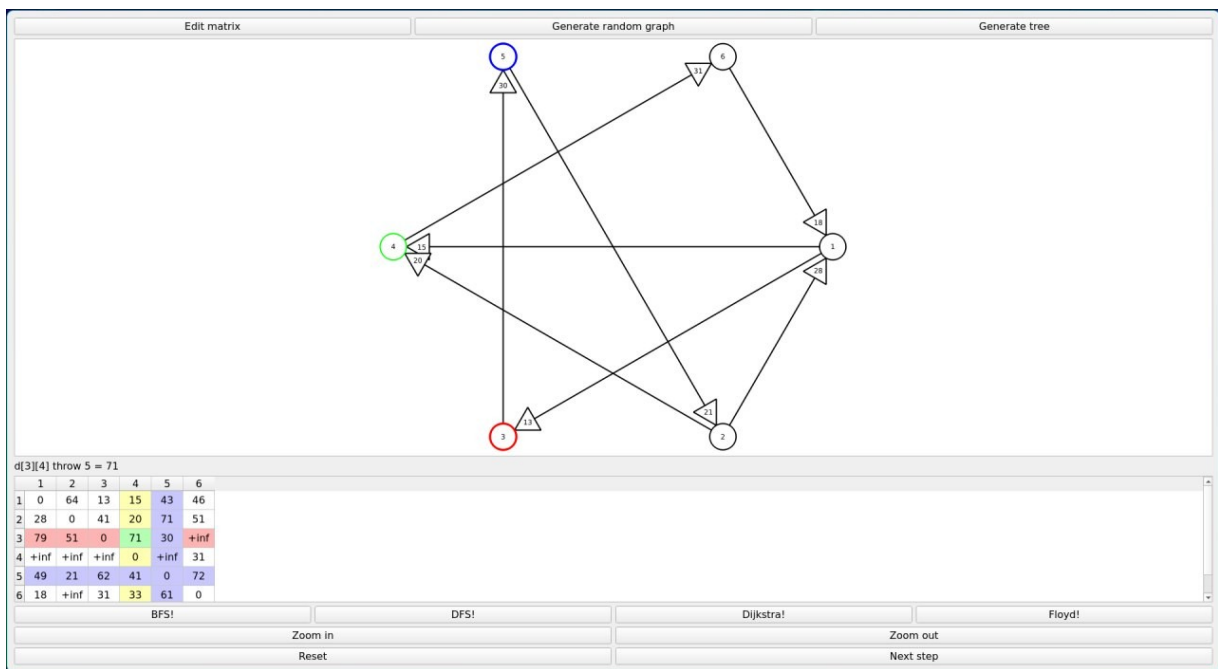
BFS. Путь: 5→2→1, вернулись в 2→4. Синим помечены used vertex, красным – текущий шаг.



DFS: 5→2→1→3, вернулись 1→4



Dijkstra: 5→2→1



Floyd: Синим помечена 5-я vertex THROW, красным – 3 FROM, желтым – 4 TO. Новое значение $[3][4]$ = сумме клеток, лежащих на пересечении красной+синей линии и желтой+синей.